

Implementation Doctrine

Local-First P2P Architecture Development Gospel

Status

Proposed implementation doctrine v1. This is the build-order authority for the first serious implementation. It is intentionally stricter than the architecture documents: when there is tension between a nice feature and this doctrine, follow this doctrine until a written architecture decision record changes it.

Core decision

Build a PWA-first product with a hybrid-ready architecture. The first user-facing client may be a PWA, but the engine, data formats, identity model, media manifests, and sync interfaces must be compatible with future full peers implemented with a native/Bare/Holepunch-capable runtime.

Prepared: May 2026

Scope: implementation order, gates, non-negotiable rules, test strategy, and release path for the local-first P2P architecture.

Contents

1. 1. Doctrine summary
2. 2. Non-negotiable implementation rules
3. 3. Product/runtime decision
4. 4. Development tracks and dependency graph
5. 5. Phase plan and exit gates
6. 6. Protocol-first object model
7. 7. Infrastructure classes and deployment discipline
8. 8. MLS private-group encryption plan
9. 9. Media, compression, presence, naming, and search milestones
10. 10. Testing, security, and quality gates
11. 11. What not to build yet
12. 12. Team workflow and decision records
13. 13. Reference basis

1. Doctrine summary

The project must be implemented as a protocol-first local-first P2P system, not as a normal server-backed web application with local caching. The first product can be a PWA, but the architecture must preserve a path to full peers, persistent availability peers, and direct P2P replication.

- **Canonical state:** signed logs, identity control events, room control records, media manifests, and source-referenced events. Derived indexes are rebuildable and never authoritative.
- **First client:** PWA light peer with local signing, local verification, local cache, encrypted payload handling, and bridge transports.
- **Full peer path:** native/Bare/Holepunch-capable runtime later implements the same interfaces with Corestore, Hypercore, Autobase, Hyperbee, Hyperdrive/Hyperblobs, Hyperswarm/HyperDHT.
- **Infrastructure:** stateless edge helpers, stateful edge actors, and persistent peers. Each class has a different responsibility and must remain non-authoritative unless explicitly designed otherwise.
- **Private groups:** MLS is the recommended candidate for serious private DM/group encryption, but only over a per-room virtual Delivery Service that sequences Commits and records control state.
- **Media:** message/post logs contain signed manifest references, not large binaries. Media uses chunked encrypted blob/file storage with optional public IPFS/CDN mirrors.
- **Search/intelligence:** local-first hybrid search combines lexical, semantic/vector, metadata, trust, permission, and provenance filters. Private search is local by default.
- **Compression:** zstd is the default durable compression profile; FastCDC/zchunk-style chunking is used for large snapshots/files; compression must be safe around encryption and side-channel risks.

Doctrine in one paragraph

Build a PWA-first light peer that creates and verifies signed local-first data, backed by non-authoritative bridge infrastructure. In parallel, keep every protocol surface compatible with a future full peer that runs the Holepunch-style stack directly. Every feature must respect cryptographic source authority, local-first rebuildability, privacy boundaries, and eventual migration from bridge-assisted web mode to full P2P mode.

2. Non-negotiable implementation rules

Rule	Requirement	Meaning
R1	No server-authoritative private state	Servers, bridges, relays, and super peers may store/forward/index signed or encrypted data. They must not become the private database of record.
R2	Every durable event is signed	Messages, identity events, name records, media manifests, index records, MLS control receipts, and moderation records require verifiable provenance.
R3	Private payloads are encrypted before untrusted storage	A bridge or seed peer may possess ciphertext and metadata required for routing. It must not possess plaintext private messages/media by default.
R4	Derived indexes are disposable	Hyperbee views, semantic indexes, feed indexes, search indexes, notifications, and public super-peer indexes must be rebuildable from

Rule	Requirement	Meaning
		signed source records.
R5	PWA is a light peer	The PWA may sign, verify, decrypt, cache, and search local data. It must not be assumed to be an always-on seeder, relay, or Delivery Service.
R6	Full peer compatibility is mandatory	All data formats and interfaces must be implementable later by a native/Bare full peer without semantic migration.
R7	MLS control state is not a CRDT	Application state can be local-first/CRDT-like. MLS Commit ordering must be linearized by a virtual Delivery Service or explicitly handled by a future DMLS-like mode.
R8	Names are not identity	Human names, handles, and NIP-05-like proofs are discovery/display hints. The cryptographic identity remains the account reference.
R9	Large media is manifest-referenced	Logs carry media manifest refs. Actual media uses chunked encrypted storage and replication policy.
R10	Presence is ephemeral	Typing indicators, recording status, upload progress, and live cursors are TTL-based and not durable app history.
R11	Compression must be bounded and safe	Enforce max decoded size, trusted dictionaries, scoped dedupe, and no unsafe compression of secrets mixed with attacker-controlled data.
R12	No phase advances without tests	Every phase must ship with unit tests, integration tests, fixtures, failure-mode tests, and a written exit record.

3. Product/runtime decision

The first production client may be a PWA because distribution friction is lower and the web is useful for onboarding, profile previews, invites, contact cards, and public discovery. The architecture must still be hybrid because a browser should not be expected to provide the full P2P behavior needed for durable seeding, DHT participation, large media processing, or long-running private indexes.

Client class	Build first?	Role	Must not be used for
PWA light peer	Yes	User-facing v1 client: local cache, local signing, local verification, encrypted payload handling, bridge sync, basic chat/social flows.	Always-on seeding, durable Delivery Service, large media seeding, full Hyperswarm peer duties.
Stateful bridge	Yes	PWA bridge, encrypted mailbox, Durable Stream-like resumable delivery, WebSocket fanout, notification hints, public index stream.	Plaintext private authority, unverified mutation source, private semantic indexing by default.
Persistent full peer / super peer	Prototype early, production after PWA basics	Long-running availability node: seed logs/media, full	User identity authority, plaintext access to private

Client class	Build first?	Role	Must not be used for
		P2P network, public indexes, optional virtual DS.	data by default.
Native/Bare client	Not first product requirement	Future full peer client with Corestore/Hypercore/Autobase/Hyperdrive/Hyperswarm capabilities.	Prerequisite for proving PWA UX.

Implementation decision

The first app can be PWA-first, but the first architecture cannot be browser-only. Build protocol interfaces now; implement them with browser adapters first; replace or augment them with full peer adapters later.

4. Development tracks and dependency graph

The work should be organized into tracks that can progress in parallel after the core contracts are frozen. The dependency graph below determines what can be built safely.

Track A: Protocol contracts

- > identity control log
- > signed event store
- > bridge protocol
- > chat/social/media/search layers

Track B: PWA light peer

- > local storage + crypto
- > bridge sync
- > chat/social UX
- > local search/cache

Track C: Infrastructure

- > stateless helpers
- > stateful edge actors
- > persistent peer prototype
- > public indexes and availability

Track D: Private group security

- > MLS library evaluation
- > virtual DS
- > control log
- > fork detection

Track E: Media/search/intelligence

- > media manifests
- > chunked storage
- > hybrid search
- > local semantic index

Track	Primary output	Earliest dependency	Blocker if skipped
A. Protocol contracts	Versioned schema, signatures, source refs, fixtures	None	Everything drifts into incompatible ad-hoc formats.
B. PWA light peer	First usable product shell	A1 event envelope, A2	No usable UI or onboarding

Track	Primary output	Earliest dependency	Blocker if skipped
		identity model	proof.
C. Infrastructure	Bridge and availability layers	A1 event envelope	PWA cannot sync/recover reliably.
D. MLS/private groups	Production-grade private group encryption candidate	A2 identity, C2 stateful bridge	Private group chat remains toy or insecure.
E. Media/search/intelligence	Real social app utility	A1 source refs, B1 local storage	App cannot scale beyond text demos.

5. Phase plan and exit gates

This is the build order. Phases may overlap only after their dependencies are satisfied. A phase is not complete until its exit gate is met and recorded.

Phase 0 - Doctrine, repo discipline, and ADRs

Build work:

- Create repository structure for protocol, PWA client, bridge, tests, docs, fixtures.
- Create architecture decision record template.
- Create threat-model template.
- Define versioning policy for schemas and storage adapters.
- Set up CI, lint, typecheck, unit test, integration test, fixture validation.

Exit gate:

- ADR-000 runtime decision exists.
- Protocol naming glossary exists.
- CI rejects untyped/untested protocol changes.
- No implementation starts without fixture strategy.

Phase 1 - Protocol primitives and canonical fixtures

Build work:

- Define SourceRef, SignedEvent, DeviceRef, IdentityRef, CompressionDescriptor, MediaManifestRef, NameBindingRef.
- Define canonical serialization rules.
- Define hash/signature verification rules.
- Create golden fixtures for valid/invalid events.
- Create property tests for canonicalization, duplicate suppression, and signature verification.

Exit gate:

- Two implementations or two independent test paths verify the same fixture hashes.
- Malformed, replayed, duplicate, and unknown-version events are rejected predictably.
- No app feature uses unversioned JSON blobs.

Phase 2 - Identity control log v1

Build work:

- Implement root identity creation for test/dev.
- Implement device.added, device.revoked, capability.granted, capability.revoked, identity checkpoint.
- Implement local verification of device delegations.
- Implement revocation policy using epochs/checkpoints, not wall-clock-only timestamps.
- Implement contact card v1.

Exit gate:

- Authorized device writes accepted.
- Revoked device writes rejected after revocation checkpoint.
- Stale/ambiguous writes are quarantined or rejected according to policy.
- PWA can import/export contact card.

Phase 3 - PWA light peer foundation

Build work:

- Implement local store adapter: IndexedDB first, OPFS optional where available.
- Implement WebCrypto signing/encryption adapter with extractability policy.
- Implement local event append/read/query.
- Implement bridge transport interface.
- Implement privacy-safe logging and error model.

Exit gate:

- PWA can create identity, add device key, append signed local events, reload, verify them, and recover local state.
- No private plaintext in diagnostic logs.
- Storage adapter can rebuild local view from event log.

Phase 4 - Bridge and infrastructure classes v1

Build work:

- Implement stateless public helpers for contact card/profile/name lookup.
- Implement stateful edge actor or equivalent bridge for encrypted mailbox and resumable stream.
- Implement persistent peer prototype or simulated seed service.
- Implement bridge response format as signed/encrypted source refs, not server-authored truth.

Exit gate:

- PWA can sync through bridge after reconnect using offsets/cursors.
- Bridge cannot forge valid source events.
- Private bridge payloads are opaque ciphertext.
- Failure of one bridge does not destroy canonical state.

Phase 5 - Chat vertical slice without production-scale complexity

Build work:

- Implement room metadata model and room event model.
- Implement message.created, message.edited, message.deleted, reaction.added/removed.

- Implement deterministic local room view.
- Implement encrypted payload placeholder only for test until MLS/private encryption lands; user-facing builds must not ship plaintext private chat.
- Implement bridge-delivered room updates.

Exit gate:

- Two PWA clients can exchange signed room events through bridge and rebuild the same view.
- Duplicate/out-of-order delivery does not duplicate messages.
- Invalid writer rejected.
- Private plaintext prohibited in production flag.

Phase 6 - MLS private group encryption v1

Build work:

- Evaluate OpenMLS and mls-rs against WASM/PWA, storage, credential validation, and future native/Bare use.
- Implement MLSKeyPackageService.
- Implement per-room virtual Delivery Service.
- Implement MLS control log: proposals, accepted commit, Welcome delivery, epoch receipts.
- Implement sequencer lease and handoff records.
- Implement fork detector and partition policy.

Exit gate:

- One room can add member, remove member, rotate epoch, and recover after reconnect.
- Only one accepted Commit per epoch.
- Forked epoch produces visible safety error.
- PWA participates as MLS client but not as durable Delivery Service.
- Large media not embedded directly in MLS application messages.

Phase 7 - Media manifests and chunked storage v1

Build work:

- Implement media manifest v1.
- Implement image pipeline: metadata stripping policy, thumbnail, preview, encrypted original.
- Implement Hyperdrive/Hyperblobs-compatible abstraction with browser fallback.
- Implement replication policy: private, group, public.
- Implement optional public mirror field for future IPFS/CDN.

Exit gate:

- Message can reference media manifest.
- Recipient fetches preview first and original on demand.
- Wrong hash rejected.
- Seed/bridge cannot decrypt private media.

Phase 8 - Ephemeral presence plane

Build work:

- Implement typing, recording, upload progress as TTL events outside durable logs.
- Implement unreliable/unordered channel where available; bridge fallback where not.
- Rate-limit presence updates.
- Ignore stale seq numbers.

Exit gate:

- Typing expires without stop event.
- Presence is not replayed after reconnect.
- No presence data becomes durable chat history.

Phase 9 - Social outbox v1

Build work:

- Implement personal public outbox.
- Implement post.created, post.edited, post.deleted, follow, reaction, comment, repost events.
- Implement reply/comment reference model by SourceRef.
- Implement local following feed and backlink index.
- Implement spam quarantine buckets.

Exit gate:

- Alice can comment on Bob post from Alice outbox.
- Bob can discover through local or bridge index.
- Client verifies source event before rendering.
- Unknown references do not pollute primary reply view.

Phase 10 - Human naming and NIP-05-like proofs

Build work:

- Implement petname book.
- Implement personal namespace zone.
- Implement community namespace record.
- Implement profile reciprocal claim.
- Implement name revocation and expiration.
- Implement anti-phishing UI states.

Exit gate:

- Saved contact resolves offline.
- damon@namespace-style binding verifies by zone signature and reciprocal claim.
- Revoked/expired names lose badge.
- Name changes never silently replace followed identity.

Phase 11 - Local search and intelligence v1

Build work:

- Implement local lexical search with field boosts.
- Implement search object model with SourceRef and permission scope.

- Implement vector/semantic index adapter interface.
- Implement ObjectBox-inspired embedding lifecycle states.
- Implement Weaviate-inspired hybrid fusion profiles.
- Implement query privacy rule: private search local by default.

Exit gate:

- Search result has source provenance.
- Private result passes current permission check before display.
- Deleted/revoked content is removed or marked stale.
- Hybrid search can combine exact name, BM25-like text, metadata, and semantic candidates.

Phase 12 - Compression, chunking, and dedupe

Build work:

- Implement CompressionDescriptor.
- Add zstd profile for durable event batches where runtime supports it; gzip fallback for browser compatibility.
- Add signed dictionary policy for repetitive events.
- Implement FastCDC-style chunking for large files/snapshots.
- Implement scoped private dedupe.

Exit gate:

- Compression bomb protections pass tests.
- Unknown dictionaries rejected.
- Private dedupe does not leak cross-scope equality.
- Large file update transfers changed chunks only in prototype.

Phase 13 - Native/Bare full peer prototype

Build work:

- Implement same interfaces with Corestore/Hypercore/Autobase/Hyperbee/Hyperdrive/Hyperswarm where feasible.
- Run compatibility fixtures between PWA bridge mode and full peer mode.
- Implement persistent seeding and direct peer replication prototype.
- Test PWA-to-bridge-to-full-peer flow.

Exit gate:

- Same signed events and manifests work in PWA and full peer.
- Full peer can seed/catch up PWA state.
- No data model fork between web and native path.

Phase 14 - Hardening and beta readiness

Build work:

- Threat model every protocol surface.
- Run malicious peer tests.

- Run network partition/reconnect tests.
- Run media abuse/resource exhaustion tests.
- Run identity revocation chaos tests.
- Audit privacy logs and telemetry.
- Finalize backup/recovery UX.

Exit gate:

- Private content leakage tests pass.
- Bridge compromise model documented and tested.
- All beta features have rollback/migration plan.
- Open risks are documented, not hidden.

6. Protocol-first object model

Implementation must begin with stable protocol objects. Application code may wrap them, but it must not bypass them. Every object listed here needs versioning, validation, canonical serialization, fixtures, and negative tests.

Object	Purpose	Must include
SignedEvent	Durable app event envelope	version, type, sourceRef, author/device, payload hash, signature, created/logical ordering metadata
SourceRef	Verifiable pointer to source data	core/log key or bridge scope, sequence/index, event hash, optional author identity
IdentityEvent	Identity control log entry	event type, target key, capability/revocation data, identity epoch, signature
Capability	Scoped delegation	issuer, audience, scope, resource, expiry, epoch, signature
RoomEvent	Chat room application event	room id, writer key, event kind, encrypted payload ref, signature
MLSControlRecord	Private group encryption control	group id, epoch, proposal/commit/welcome ref, sequencer receipt, previous hash
MediaManifest	Large media descriptor	media id, variants, hashes, encryption info, chunk refs, replication policy, signature
NameBinding	Human name to identity mapping	zone key, label, target identity/profile/discovery refs, expiry, sequence, signature
SearchObject	Search/index projection	sourceRef, fields, named vectors, permission scope, lifecycle/model metadata
CompressionDescriptor	Compression/chunking metadata	algorithm, dictionary, chunking mode, sizes, hashes, max decoded size

Schema rule

Unknown major versions must be rejected. Unknown optional fields may be ignored only if the object explicitly permits extension. Every security-sensitive object must be canonicalized before hashing/signing.

7. Infrastructure classes and deployment discipline

Infrastructure must be classified before implementation. The class determines what state it may hold, how it is tested, and what compromise means.

Class	Examples	Allowed responsibilities	Forbidden responsibilities
Class 1: Stateless edge helpers	Static/edge profile fetch, contact-card fetch, namespace lookup, manifest lookup	Return signed public records or short-lived upload/download helpers.	Live room sequencing, private plaintext handling, canonical state ownership.
Class 2: Stateful edge actors	Durable Object-like bridge, WebSocket fanout, resumable stream actor, encrypted mailbox actor	Coordinate bridge sessions, maintain offsets, temporary encrypted mailboxes, small-room virtual DS if policy allows.	Long-term user database authority, private semantic indexing by default, unrestricted media seeding.
Class 3: Persistent peers	Super peer, seed peer, native/Bare full peer, public indexer, media seeder	Seed logs/media, run full P2P stack, index public data, bridge clients, virtual DS for rooms.	Identity authority, plaintext private state by default, irreversible user lock-in.

A Nostr-like relay is a useful mental model only for the simplest non-authoritative event relay. Our richer architecture also needs room control, identity revocation, media manifests, public indexes, MLS control logs, and bridge resumability. Those require more discipline than a dumb event store.

8. MLS private-group encryption plan

MLS is the recommended candidate for serious private DM/group encryption, but it must be implemented as a module over a virtual Delivery Service. MLS group state advances through a linear sequence of epochs; Commit messages must be ordered. The P2P/local-first layer may deliver candidate messages over many transports, but the MLS control plane still needs one accepted Commit for each epoch.

MLS module	Responsibility	Implementation gate
KeyPackageService	Publish/fetch one-time KeyPackages and last-resort fallback packages without stuffing rotating inventory into public name records.	Package reuse prevention, freshness checks, authentication binding.
CredentialService	Bind device identity/capability state to MLS credentials.	Reject revoked/stale devices; support future native hardware-backed keys.
VirtualDeliveryService	Per-room ordering service for MLS control plane.	Exactly one accepted Commit per epoch; auditable receipt.
MLSControlLog	Append-only record of proposals, commits, welcomes, epoch receipts, handoffs, fork alerts.	Clients can rebuild control state and detect equivocation.
SequencerLease	Defines active sequencer and handoff policy.	Expired/unavailable sequencer can be replaced by policy without silent fork.
ForkDetector	Detect conflicting accepted commits or tree/member hash divergence.	Pause membership changes and surface safety error.

- Application messages can be local-first above MLS, but MLS Commit ordering itself is not a CRDT merge problem.
- During partition, allow safe application messages under the current epoch only if policy permits; queue proposals; block membership-changing Commits until a sequencer is reachable unless a future DMLS-like mode is implemented.
- Large media should use manifest-referenced encrypted blobs. MLS may protect media-key distribution or epoch-derived wrapping keys, not carry the whole media binary.
- PWA clients may participate in MLS as clients. They should not be assumed to provide durable Delivery Service behavior.

9. Media, compression, presence, naming, and search milestones

Subsystem	First useful milestone	Never do this
Media	Image attachment with signed manifest, thumbnail, encrypted original, bridge/super-peer opaque storage.	Do not put large binaries directly in durable chat/social events.
Presence	Typing/recording/upload progress over TTL ephemeral channel.	Do not persist typing indicators in Hypercore/Autobase or replay them after reconnect.
Naming	Contact cards, petnames, personal namespace, community name binding, reciprocal claim.	Do not treat global first-claim names as identity.
Search	Lexical local search with SourceRef provenance, then hybrid semantic search.	Do not send private query text or private embeddings to remote indexes by default.
Compression	CompressionDescriptor + zstd/gzip support + decoded-size limits.	Do not accept unknown dictionaries or unbounded decompression.

10. Testing, security, and quality gates

Testing is part of the protocol. The system is not safe if it only passes happy-path browser demos.

Test category	Required examples
Canonicalization	Same object produces same hash across adapters; field order and unknown fields handled predictably.
Signature and revocation	Valid signature accepted; wrong signer rejected; revoked device rejected; stale delegation quarantined.
Bridge compromise	Bridge returns forged event, stale event, reordered event, duplicate event, missing event; client behavior is correct.
Network failure	Disconnect/reconnect, offset resume, duplicate packet, dropped mailbox ACK, unavailable sequencer.
MLS fork	Two commits for same epoch, stale Welcome, removed member sends message, sequencer handoff race.
Media abuse	Wrong hash, oversized manifest, corrupt chunk, decompression bomb, private media visible to seed peer.
Search privacy	Private result never returned from public index; revoked content not searchable; query text not sent remotely by default.
Compression safety	Max decoded size enforced; unknown dictionary rejected; private dedupe scoped.

Release rule

A feature is not beta-ready until its malicious-input tests pass. A feature that works only when all peers are honest and online is a demo, not an implementation.

11. What not to build yet

- **Do not build a full native app before the protocol contracts exist.** Native/Bare full peer work should validate the contracts, not invent new ones.
- **Do not build a global username registry.** Start with petnames, contact cards, personal namespaces, and community namespaces.
- **Do not make IPFS the default private media layer.** Keep IPFS as optional public immutable mirror until privacy and lifecycle needs are proven.
- **Do not build public semantic search before local/private search.** Private query privacy and source provenance must be solved first.
- **Do not implement DMLS/fork-resilient offline commits in v1.** Use standard MLS with a virtual Delivery Service first.
- **Do not compress everything blindly.** Compression must be scope-aware, bounded, and safe around encryption.
- **Do not implement staking/token reputation as spam v1.** Use relationship filters, signed moderation logs, public labels, and quarantine first.
- **Do not let the bridge become the backend.** The bridge delivers signed/encrypted data and public indexes. It does not define truth.

12. Team workflow and decision records

The project needs a disciplined engineering process because the system spans cryptography, distributed systems, local-first UX, browser constraints, and future native/full-peer compatibility.

Artifact	Required before
ADR	Any change to runtime, storage, identity, MLS, bridge, media, search, compression, or naming model.
Protocol spec	Any code writes durable events or accepts remote events.
Fixture pack	Any protocol object is considered implemented.
Threat model note	Any feature stores/forwards private data, accepts remote input, or changes key/identity state.
Migration note	Any schema/index/storage version changes.
Exit report	Any phase is declared complete.

- Every protocol change increments a version and updates fixtures.
- Every adapter must pass the shared conformance suite.
- Every bridge response must be modeled as untrusted input.
- Every private feature must document what metadata leaks remain.
- Every derived index must document rebuild conditions and deletion behavior.
- No duplicate protocol concepts under different names; if a new object looks similar to an existing object, write an ADR before adding it.

13. Reference basis

These references informed the doctrine. They do not imply that every referenced system is adopted wholesale. The project uses them as implementation and architecture references only where explicitly stated.

ID	Reference	URL	Why it matters
R1	Pear Docs	https://docs.pears.com/	Pear is the P2P runtime reference and lists the Holepunch building blocks used in the full-peer path.
R2	Holepunch GitHub organization	https://github.com/holepunchto	Reference source for Hypercore, Corestore, Autobase, Hyperswarm, HyperDHT ecosystem components.
R3	Hypercore replication with Hyperswarm	https://docs.pears.com/howto/replicate-and-persist-with-hypercore.html	Reference for Hypercore persistence and peer replication examples.
R4	MLS RFC 9420	https://www.rfc-editor.org/rfc/rfc9420.html	Reference for MLS epochs, Delivery Service assumptions, KeyPackages, and group security model.
R5	MLS Architecture RFC 9750	https://www.rfc-editor.org/rfc/rfc9750.html	Reference for MLS AS/DS architecture and strongly/eventually consistent Delivery Service distinctions.
R6	DMLS draft	https://www.ietf.org/archive/id/draft-kohbrok-mls-dmls-01.html	Reference for decentralized MLS ordering/fork handling and forward secrecy caveats.
R7	Electric Durable Streams announcement	https://electric.ax/blog/2025/12/09/announcing-durable-streams	Reference for reliable, resumable HTTP client delivery streams.
R8	Cloudflare Durable Objects docs	https://developers.cloudflare.com/durable-objects/	Reference for stateful serverless coordination and actor-like edge infrastructure.
R9	ObjectBox on-device/vector positioning	https://objectbox.io/	Reference for on-device/offline-first local database and edge AI design inspiration.
R10	Weaviate hybrid search docs	https://docs.weaviate.io/weaviate/search/hybrid	Reference for vector + BM25F hybrid search and fusion weighting.
R11	Zstandard RFC 8878	https://datatracker.ietf.org/doc/html/rfc8878	Reference for zstd format/content encoding.
R12	Zstandard project	https://facebook.github.io/zstd/	Reference for zstd performance, dictionary compression, and implementation characteristics.
R13	FastCDC USENIX paper	https://www.usenix.org/conference/atc16/technical-sessions/presentation/xia	Reference for content-defined chunking and deduplication motivation.
R14	Nostr NIP-01	https://github.com/nostr-protocol/nips/blob/master/01.md	Reference for simple signed event relay model used only as a non-authoritative relay

ID	Reference	URL	Why it matters
			comparison.

Appendix A - Phase checklist

Phase	Completion checklist
Phase 0 - Doctrine, repo discipline, and ADRs	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 1 - Protocol primitives and canonical fixtures	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 2 - Identity control log v1	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 3 - PWA light peer foundation	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 4 - Bridge and infrastructure classes v1	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 5 - Chat vertical slice without production-scale complexity	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 6 - MLS private group encryption v1	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 7 - Media manifests and chunked storage v1	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 8 - Ephemeral presence plane	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 9 - Social outbox v1	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 10 - Human naming and NIP-05-like proofs	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 11 - Local search and intelligence v1	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 12 - Compression, chunking, and dedupe	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 13 - Native/Bare full peer prototype	Spec written; fixtures created; tests pass; exit report accepted; docs updated.
Phase 14 - Hardening and beta readiness	Spec written; fixtures created; tests pass; exit report accepted; docs updated.

Appendix B - Glossary

Term	Meaning in this doctrine
Light peer	A client that signs/verifies/caches/decrypts locally but relies on bridge infrastructure for sync and does not provide always-on seeding.
Full peer	A client or node that can participate directly in P2P replication and local storage using the full engine path.
Bridge	Non-authoritative infrastructure that transports signed/encrypted state between clients and peers.
Virtual Delivery Service	Per-room MLS control-plane service that orders Commits and delivers control messages without decrypting content.
SourceRef	A stable verifiable pointer to a signed source event or record.
Derived index	A local/public index rebuilt from source records; never canonical.

Term	Meaning in this doctrine
Namespace	A signed zone mapping human labels to cryptographic identities or resources.